

Bloc 2 TP Curseurs PL/SQL

Contexte	2
Mise en place sur phpMyAdmin	2
Partie 1 : MediaTek	6
Étape 0 : Préparation	15
Exercice 1 : Curseur implicite	16
Exercice 2 : Curseur explicite	23
Exercice 3 : Casser le curseur	26
Exercice 4 : Trois façons de boucler	30
Exercice 5 : Curseurs paramétrés	39
Exercice 6 : FOR UPDATE	46
Exercice 7 : Mini-projet	55
Compte rendu final	60
Conclusion	61

Contexte

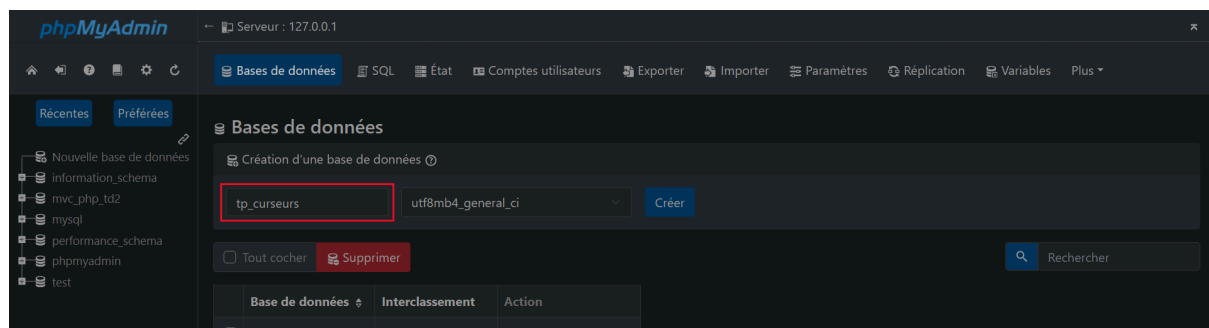
Ce TP va nous faire découvrir les **curseurs** en **PL/SQL**. La première partie reprend la base **MediaTek** (clients, produits, factures) avec deux fonctions stockées et une procédure, la deuxième partie fait expérimenter les curseurs avec le cycle **prédire -> taper -> observer -> conclure** : curseur implicite, curseur explicite et ses attributs, les trois boucles, les curseurs paramétrés, les verrous avec **FOR UPDATE** et un mini-projet.

Le TP est prévu pour **Oracle** (SQL Developer ou SQL*Plus), mais nous on travaille avec **phpMyAdmin** sous **XAMPP**, donc avec **MariaDB 10.4**. MariaDB a un mode Oracle (sql_mode = ORACLE) qui accepte une bonne partie de la syntaxe PL/SQL (DECLARE, %TYPE, curseurs, %ISOPEN, %FOUND...), donc on a pu garder presque tout le code du TP. Quand ça ne marchait pas pareil qu'Oracle on l'a noté dans l'exercice.

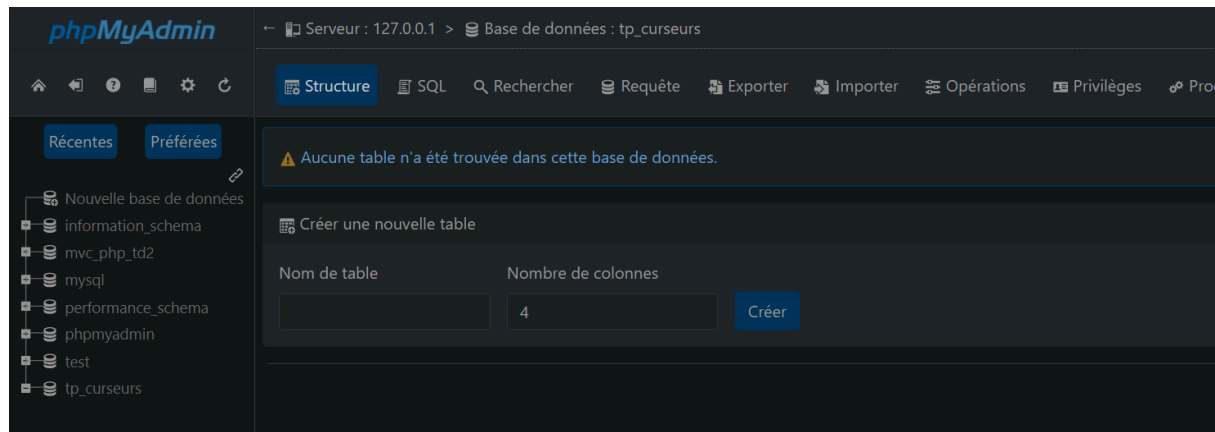
Les scripts sont écrits dans **VS Code** puis collés dans l'onglet **SQL** de phpMyAdmin. *(Les scripts sont dans le dossier SQL rendu avec le compte rendu).*

Mise en place sur phpMyAdmin

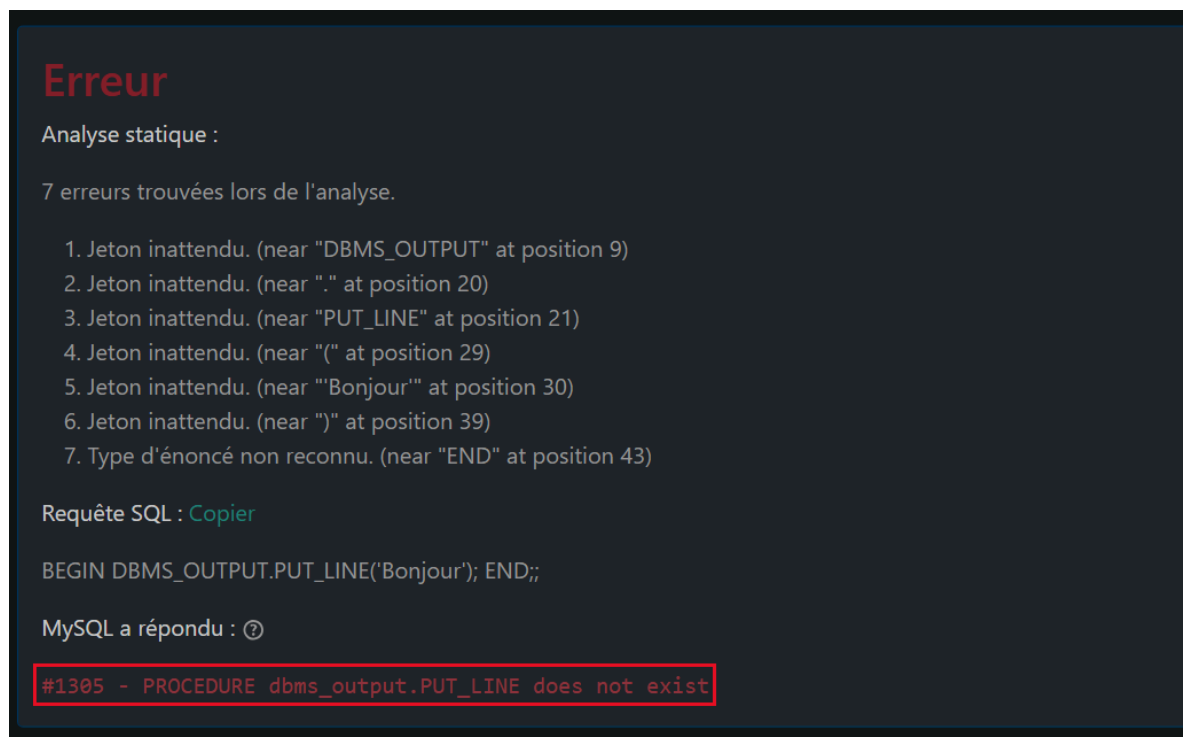
On crée une nouvelle base "**tp_curseurs**" dans l'onglet "**Bases de données**" :



La base est créée et vide :

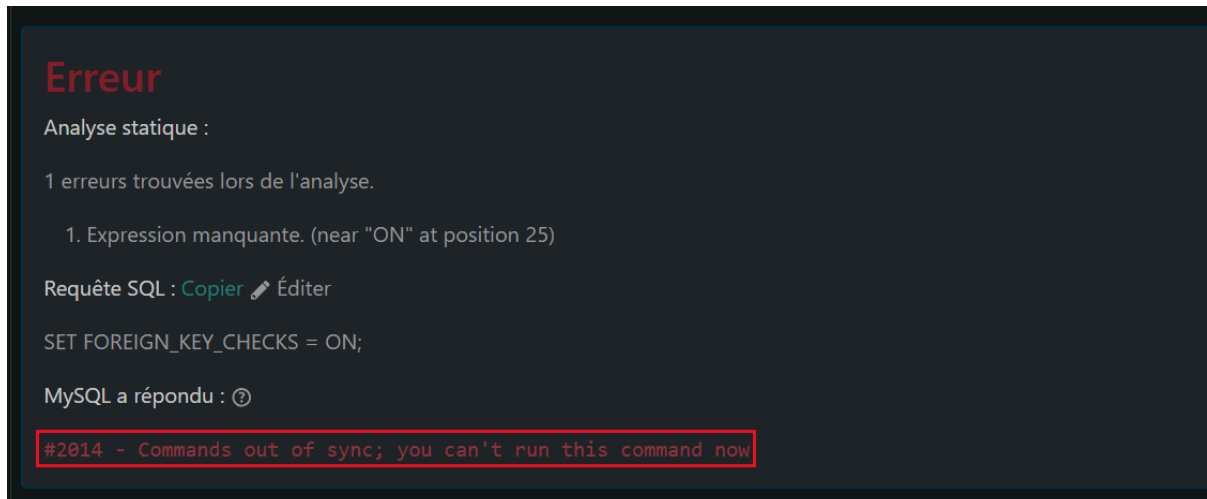


Pour afficher du texte, le TP utilise **DBMS_OUTPUT.PUT_LINE** avec **SET SERVEROUTPUT ON**. On a testé avec un petit bloc en mode Oracle, mais ce package n'existe pas sur MariaDB :



L'analyse statique de phpMyAdmin ne connaît pas la syntaxe PL/SQL et met des erreurs partout (ça le fera sur tous les blocs), c'est la réponse de MySQL en bas qui compte.

Deuxième essai : faire des **SELECT** directement dans le bloc pour afficher les valeurs. Cette fois c'est phpMyAdmin qui plante dès qu'un bloc renvoie plusieurs résultats :



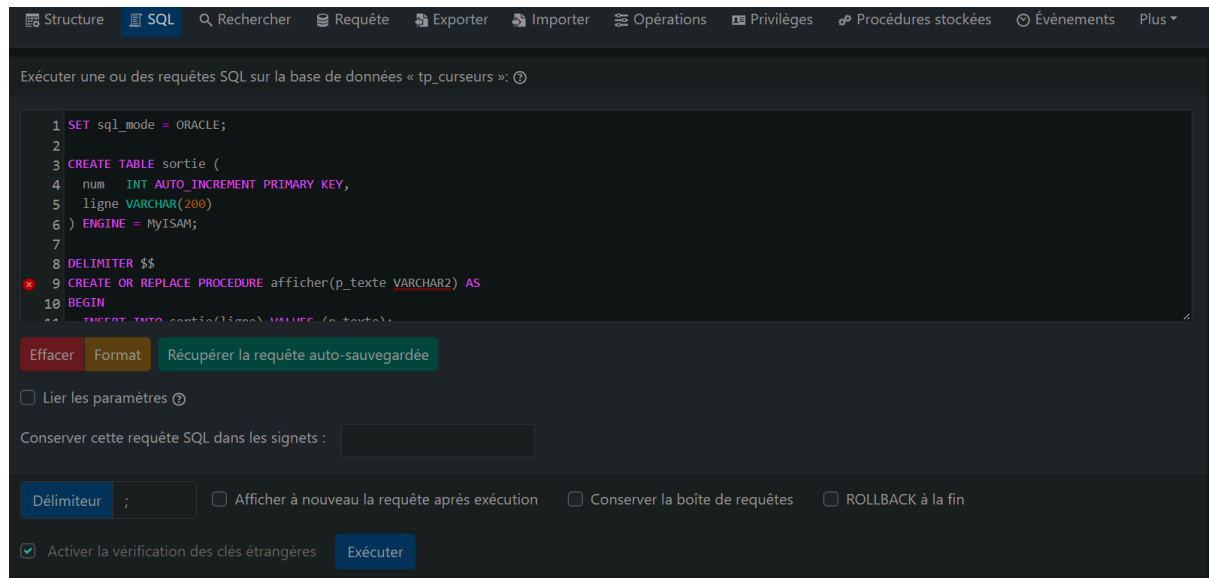
La solution qu'on a trouvé : une table **sortie** où on range les lignes à afficher, et une petite procédure **afficher()** qui remplace DBMS_OUTPUT.PUT_LINE. À la fin de chaque script on fait un SELECT sur cette table. Elle est en **MyISAM** pour que les lignes ne soient pas annulées par un ROLLBACK :

```
SET sql_mode = ORACLE;

CREATE TABLE sortie (
  num INT AUTO_INCREMENT PRIMARY KEY,
  ligne VARCHAR(200)
) ENGINE = MyISAM;

DELIMITER $$
CREATE OR REPLACE PROCEDURE afficher(p_texte VARCHAR2) AS
BEGIN
  INSERT INTO sortie(ligne) VALUES (p_texte);
END;
$$
DELIMITER ;
```

On colle le script dans l'onglet SQL. Pour les blocs il faut **DELIMITER \$\$**, sinon phpMyAdmin coupe le bloc à chaque point-virgule :



Tout passe :



Partie 1 : MediaTek

On crée les tables à partir du schéma E-A. L'attribut Date de Facture devient **date_facture** (DATE est un type) et l'association Concerne devient une table avec une clé primaire composée et la **quantite** :

```
SET sql_mode = ORACLE;

CREATE TABLE client (
  num          NUMBER(5) PRIMARY KEY,
  nom          VARCHAR2(30),
  prenom       VARCHAR2(30),
  adresse      VARCHAR2(80),
  date_naissance DATE,
  tel          VARCHAR2(14),
  sexe         CHAR(1)
);

CREATE TABLE produit (
  num          NUMBER(5) PRIMARY KEY,
  designation   VARCHAR2(50),
  prix         NUMBER(8,2),
  stock        NUMBER(5)
);

CREATE TABLE facture (
  num          NUMBER(5) PRIMARY KEY,
  date_facture DATE,
  num_client   NUMBER(5) NOT NULL,
  CONSTRAINT fk_facture_client FOREIGN KEY (num_client) REFERENCES client(num)
);

CREATE TABLE concerne (
  num_facture  NUMBER(5),
  num_produit  NUMBER(5),
  quantite     NUMBER(3),
  CONSTRAINT pk_concerne PRIMARY KEY (num_facture, num_produit),
  CONSTRAINT fk_concerne_facture FOREIGN KEY (num_facture) REFERENCES facture(num),
  CONSTRAINT fk_concerne_produit FOREIGN KEY (num_produit) REFERENCES produit(num)
);
```

Puis un jeu de données : 5 clients (Emma Petit n'a aucune facture, pour tester le cas vide), 6 produits et 10 factures :

```
INSERT INTO client VALUES (1, 'Dupont', 'Marie', '12 rue des Lilas, Lyon', DATE '1995-04-12', '06 12 34 56 78', 'F');
INSERT INTO client VALUES (2, 'Lefebvre', 'Thomas', '3 place Bellecour, Lyon', DATE '1988-11-02', '06 98 76 54 32', 'M');
INSERT INTO client VALUES (3, 'Moreau', 'Julie', '45 avenue Jean Jaurès, Villeurbanne', DATE '2001-07-23', '07 11 22 33 44', 'F');
INSERT INTO client VALUES (4, 'Bernard', 'Lucas', '8 rue Victor Hugo, Bron', DATE '1979-01-30', '06 55 44 33 22', 'M');
INSERT INTO client VALUES (5, 'Petit', 'Emma', '27 rue de la République, Lyon', DATE '2003-09-15', '07 66 77 88 99', 'F');

INSERT INTO produit VALUES (1, 'DVD Inception', 12.99, 40);
INSERT INTO produit VALUES (2, 'Blu-ray Dune', 19.99, 25);
INSERT INTO produit VALUES (3, 'CD Daft Punk - Discovery', 15.50, 30);
INSERT INTO produit VALUES (4, 'Livre SQL pour les nuls', 24.90, 15);
INSERT INTO produit VALUES (5, 'BD Astérix', 11.95, 50);
INSERT INTO produit VALUES (6, 'Casque audio', 49.90, 10);

INSERT INTO facture VALUES (1, DATE '2026-01-05', 1);
INSERT INTO facture VALUES (2, DATE '2026-01-18', 2);
INSERT INTO facture VALUES (3, DATE '2026-02-02', 1);
INSERT INTO facture VALUES (4, DATE '2026-02-14', 3);
INSERT INTO facture VALUES (5, DATE '2026-03-01', 1);
INSERT INTO facture VALUES (6, DATE '2026-03-22', 4);
INSERT INTO facture VALUES (7, DATE '2026-04-09', 2);
INSERT INTO facture VALUES (8, DATE '2026-05-17', 3);
INSERT INTO facture VALUES (9, DATE '2026-06-03', 1);
INSERT INTO facture VALUES (10, DATE '2026-06-28', 4);

INSERT INTO concerne VALUES (1, 1, 2);
INSERT INTO concerne VALUES (1, 3, 1);
INSERT INTO concerne VALUES (2, 4, 1);
INSERT INTO concerne VALUES (3, 2, 1);
INSERT INTO concerne VALUES (3, 5, 3);
INSERT INTO concerne VALUES (4, 6, 1);
INSERT INTO concerne VALUES (5, 5, 2);
INSERT INTO concerne VALUES (6, 1, 1);
INSERT INTO concerne VALUES (6, 2, 1);
INSERT INTO concerne VALUES (6, 3, 1);
INSERT INTO concerne VALUES (7, 6, 1);
INSERT INTO concerne VALUES (7, 3, 2);
INSERT INTO concerne VALUES (8, 4, 2);
INSERT INTO concerne VALUES (9, 1, 1);
INSERT INTO concerne VALUES (10, 5, 1);
INSERT INTO concerne VALUES (10, 4, 1);
```

Les tables sont bien là (la table sortie est la seule en MyISAM) :

The screenshot shows the phpMyAdmin interface for a database named 'tp_curseurs'. The left sidebar shows the database structure, including tables 'client', 'concerne', 'facture', 'produit', and 'sortie'. The main panel displays the 'Structure' tab for the 'tp_curseurs' database. It lists the following tables:

Table	Action	Lignes	Type	Interclassement	Taille	Perte
client	Parcourir, Structure, Rechercher, Insérer, Vider, Supprimer	5	InnoDB	utf8mb4_general_ci	16,0 kio	-
concerne	Parcourir, Structure, Rechercher, Insérer, Vider, Supprimer	16	InnoDB	utf8mb4_general_ci	32,0 kio	-
facture	Parcourir, Structure, Rechercher, Insérer, Vider, Supprimer	10	InnoDB	utf8mb4_general_ci	32,0 kio	-
produit	Parcourir, Structure, Rechercher, Insérer, Vider, Supprimer	6	InnoDB	utf8mb4_general_ci	16,0 kio	-
sortie	Parcourir, Structure, Rechercher, Insérer, Vider, Supprimer	0	MyISAM	utf8mb4_general_ci	1,0 kio	-
5 tables Somme		37	InnoDB	utf8mb4_general_ci	97,0 kio	0 o

Q1 Écrivez deux fonctions stockées qui prendront en argument un numéro de client, et retourneront sous forme de number le nombre de factures et le chiffre d'affaire, respectivement :

nb_factures compte les factures du client, **chiffre_affaire** fait la somme quantité x prix sur toutes ses factures. Le **NVL** renvoie 0 au lieu de NULL pour un client sans facture :

```
SET sql_mode = ORACLE;

DELIMITER $$
CREATE OR REPLACE FUNCTION nb_factures(p_num_client NUMBER) RETURN NUMBER AS
  v_nb NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_nb
  FROM facture
  WHERE num_client = p_num_client;
  RETURN v_nb;
END;
$$

CREATE OR REPLACE FUNCTION chiffre_affaire(p_num_client NUMBER) RETURN NUMBER AS
  v_ca NUMBER;
BEGIN
  SELECT NVL(SUM(c.quantite * p.prix), 0) INTO v_ca
  FROM facture f
  JOIN concerne c ON c.num_facture = f.num
  JOIN produit p ON p.num = c.num_produit
  WHERE f.num_client = p_num_client;
  RETURN v_ca;
END;
$$
DELIMITER ;

SELECT num, nom, nb_factures(num) AS nb_factures, chiffre_affaire(num) AS ca
FROM client;
```

On teste les deux fonctions avec un SELECT sur la table client :

✓ Affichage des lignes 0 - 4 (total de 5, traitement en 0,0017 seconde(s).)

```
SELECT num, nom, nb_factures(num) AS nb_factures, chiffre_affaire(num) AS ca FROM
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

	num	nom	nb_factures	ca
☐ Éditer Copier Supprimer	1	Dupont	4	134.21
☐ Éditer Copier Supprimer	2	Lefebvre	2	105.8
☐ Éditer Copier Supprimer	3	Moreau	2	99.7
☐ Éditer Copier Supprimer	4	Bernard	2	85.33
☐ Éditer Copier Supprimer	5	Petit	0	0

Le client 5 renvoie bien 0 / 0. MariaDB transforme le NUMBER du RETURN en **double**, c'est pour ça que 105.80 s'affiche 105.8.

Q2 Écrivez un programme PL/SQL anonyme qui affiche pour le client numéro 1 le nombre de factures et le chiffre d'affaire :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  v_client client.num%TYPE := 1;
BEGIN
  afficher('client ' || v_client);
  afficher('Nombre de factures : ' || nb_factures(v_client));
  afficher('Chiffre d'affaire : ' || chiffre_affaire(v_client) || ' €');
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

✓ Affichage des lignes 0 - 2 (total de 3, traitement en 0,0003 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
 [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

				ligne
☐	Éditer	Copier	Supprimer	Client 1
☐	Éditer	Copier	Supprimer	Nombre de factures : 4
☐	Éditer	Copier	Supprimer	Chiffre d'affaire : 134.21 €

Q3 Écrivez un second programme PL/SQL qui affiche pour chaque client son nombre de factures et son chiffre d'affaire sous la forme demandée :

On parcourt les clients avec un curseur dans une boucle **FOR** et on appelle les deux fonctions de la Q1 :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_clients IS SELECT num FROM client ORDER BY num;
BEGIN
  FOR r IN c_clients LOOP
    afficher('Client ' || r.num);
    afficher(nb_factures(r.num) || ' / ' || chiffre_affaire(r.num));
    afficher('-----');
  END LOOP;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

✓ Affichage des lignes 0 - 14 (total de 15, traitement en 0,0003 seconde(s))

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage

[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher

Nombre de lignes :

25

Filtrer les lignes:

Chercher dans cette

Options supplémentaires

← T →				ligne
☐	Éditer	Copier	Supprimer	Client 1
☐	Éditer	Copier	Supprimer	4 / 134.21
☐	Éditer	Copier	Supprimer	-----
☐	Éditer	Copier	Supprimer	Client 2
☐	Éditer	Copier	Supprimer	2 / 105.8
☐	Éditer	Copier	Supprimer	-----
☐	Éditer	Copier	Supprimer	Client 3
☐	Éditer	Copier	Supprimer	2 / 99.7
☐	Éditer	Copier	Supprimer	-----
☐	Éditer	Copier	Supprimer	Client 4
☐	Éditer	Copier	Supprimer	2 / 85.33
☐	Éditer	Copier	Supprimer	-----
☐	Éditer	Copier	Supprimer	Client 5
☐	Éditer	Copier	Supprimer	0 / 0
☐	Éditer	Copier	Supprimer	-----

Q4 Transformez le bloc anonyme en procédure. Testez cette procédure :

Le DECLARE est remplacé par **CREATE OR REPLACE PROCEDURE** `etat_clients` AS,
puis on l'appelle avec **CALL** :

```
SET sql_mode = ORACLE;

DELIMITER $$
CREATE OR REPLACE PROCEDURE etat_clients AS
  CURSOR c_clients IS SELECT num FROM client ORDER BY num;
BEGIN
  FOR r IN c_clients LOOP
    afficher('client ' || r.num);
    afficher(nb_factures(r.num) || ' / ' || chiffre_affaire(r.num));
    afficher('-----');
  END LOOP;
END;
$$
DELIMITER ;

TRUNCATE TABLE sortie;
CALL etat_clients();
SELECT ligne FROM sortie ORDER BY num;
```

✓ Affichage des lignes 0 - 14 (total de 15, traitement en 0,0006 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage

[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher

Nombre de lignes :

25

Filtrer les lignes:

Chercher dans cette table Trier par

Options supplémentaires

				ligne
☐	Éditer	Copier	Supprimer	Client 1
☐	Éditer	Copier	Supprimer	4 / 134.21
☐	Éditer	Copier	Supprimer	-----
☐	Éditer	Copier	Supprimer	Client 2
☐	Éditer	Copier	Supprimer	2 / 105.8
☐	Éditer	Copier	Supprimer	-----
☐	Éditer	Copier	Supprimer	Client 3
☐	Éditer	Copier	Supprimer	2 / 99.7
☐	Éditer	Copier	Supprimer	-----
☐	Éditer	Copier	Supprimer	Client 4
☐	Éditer	Copier	Supprimer	2 / 85.33
☐	Éditer	Copier	Supprimer	-----
☐	Éditer	Copier	Supprimer	Client 5
☐	Éditer	Copier	Supprimer	0 / 0
☐	Éditer	Copier	Supprimer	-----

La procédure apparaît dans l'onglet "**Procédures stockées**" avec les deux fonctions et afficher :

The screenshot shows a database management tool interface. At the top, there is a navigation bar with icons and labels for 'Structure', 'SQL', 'Rechercher', 'Requête', 'Exporter', 'Importer', and 'Opérations'. Below this, the main section is titled 'Procédures stockées' with a gear icon and a help icon. Under the title, there are three buttons: 'Tout cocher', 'Exporter', and 'Supprimer'. Below these buttons is a table with four rows of database objects. The first row is 'afficher' (PROCEDURE). The second row is 'chiffre_affaire' (FUNCTION) with a return type of 'double'. The third row is 'etat_clients' (PROCEDURE), which is highlighted with a red rectangle. The fourth row is 'nb_factures' (FUNCTION) with a return type of 'double'. Each row has a checkbox on the left and a set of action buttons on the right: 'Éditer', 'Exécuter', 'Exporter', and 'Supprimer'.

	Nom	Type	Retourne				
☐	afficher	PROCEDURE		Éditer	Exécuter	Exporter	Supprimer
☐	chiffre_affaire	FUNCTION	double	Éditer	Exécuter	Exporter	Supprimer
☐	etat_clients	PROCEDURE		Éditer	Exécuter	Exporter	Supprimer
☐	nb_factures	FUNCTION	double	Éditer	Exécuter	Exporter	Supprimer

Étape 0 : Préparation

On exécute le script du TP. On a juste rajouté les **DROP TABLE** au début pour pouvoir le relancer (on s'en sert plusieurs fois pour remettre les données de départ) et enlevé SET SERVEROUTPUT ON :

```
SET sql_mode = ORACLE;

DROP TABLE IF EXISTS employes;
DROP TABLE IF EXISTS departements;

CREATE TABLE departements (dept_id NUMBER PRIMARY KEY, dept_nom VARCHAR2(30));
CREATE TABLE employes (
  emp_id NUMBER PRIMARY KEY, nom VARCHAR2(30), salaire NUMBER(8,2),
  prime NUMBER(8,2), dept_id NUMBER REFERENCES departements(dept_id),
  date_embauche DATE);

INSERT INTO departements VALUES (10,'Compta');
INSERT INTO departements VALUES (20,'RH');
INSERT INTO departements VALUES (30,'IT');
INSERT INTO departements VALUES (40,'Archives');

INSERT INTO employes VALUES (1,'Martin',3200,500,10,DATE '2015-03-01');
INSERT INTO employes VALUES (2,'Durand',2800,NULL,10,DATE '2018-06-15');
INSERT INTO employes VALUES (3,'Petit',2800,300,10,DATE '2020-01-10');
INSERT INTO employes VALUES (4,'Leroy',1900,NULL,10,DATE '2022-09-01');
INSERT INTO employes VALUES (5,'Moreau',2500,200,20,DATE '2017-04-20');
INSERT INTO employes VALUES (6,'Simon',2100,NULL,20,DATE '2019-11-05');
INSERT INTO employes VALUES (7,'Laurent',1700,100,20,DATE '2023-02-14');
INSERT INTO employes VALUES (8,'Garcia',4100,900,30,DATE '2014-08-30');
INSERT INTO employes VALUES (9,'Roux',3600,NULL,30,DATE '2016-12-01');
INSERT INTO employes VALUES (10,'Fournier',3000,400,30,DATE '2021-05-17');
INSERT INTO employes VALUES (11,'Girard',2200,NULL,30,DATE '2024-01-08');
INSERT INTO employes VALUES (12,'Bonnet',1800,NULL,30,DATE '2024-10-21');
COMMIT;
```

Les 12 employés sont bien insérés :

✓ Affichage des lignes 0 - 11 (total de 12, traitement en 0,0003 seconde(s).)

```
SELECT * FROM employees;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Au

Options supplémentaires

	emp_id	nom	salaire	prime	dept_id	date_embauche
☐ Éditer ☐ Copier ☐ Supprimer	1	Martin	3200.00	500.00	10	2015-03-01 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	2	Durand	2800.00	NULL	10	2018-06-15 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	3	Petit	2800.00	300.00	10	2020-01-10 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	4	Leroy	1900.00	NULL	10	2022-09-01 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	5	Moreau	2500.00	200.00	20	2017-04-20 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	6	Simon	2100.00	NULL	20	2019-11-05 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	7	Laurent	1700.00	100.00	20	2023-02-14 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	8	Garcia	4100.00	900.00	30	2014-08-30 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	9	Roux	3600.00	NULL	30	2016-12-01 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	10	Fournier	3000.00	400.00	30	2021-05-17 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	11	Girard	2200.00	NULL	30	2024-01-08 00:00:00
☐ Éditer ☐ Copier ☐ Supprimer	12	Bonnet	1800.00	NULL	30	2024-10-21 00:00:00

Exercice 1 : Curseur implicite

1a. Prédisez, puis testez :

Prédiction : le département 20 (RH) a 3 employés (Moreau, Simon et Laurent), donc on s'attend à "Lignes : 3".

Résultat :

✓ Affichage des lignes 0 - 0 (total de 1, traitement en 0,0002 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

	ligne
☐ Éditer ☐ Copier ☐ Supprimer	Lignes : 0

On a eu **Lignes : 0**, on a pas compris tout de suite. En fait MariaDB compte seulement les lignes **vraiment modifiées**, et salaire = salaire ne change rien. Oracle lui compte les lignes trouvées par le WHERE.

On a refait le test avec salaire + 1 et cette fois on a bien 3 :

✓ Affichage des lignes 0 - 0 (total de 1, traitement en 0,0002 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

	ligne
☐ Éditer ☐ Copier ☐ Supprimer	Lignes : 3

(Le Warning #1196 qui apparaît à chaque fois c'est normal, c'est à cause de la table sortie en MyISAM qui ne peut pas être annulée.)

Mais en vérifiant, les salaires avaient vraiment augmenté de 1 € malgré le ROLLBACK :

✓ Affichage des lignes 0 - 2 (total de 3, traitement en 0,0004 seconde(s).)

```
SELECT emp_id, nom, salaire FROM employes WHERE dept_id = 20;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé :

Options supplémentaires

			emp_id	nom	salaire	
☐	✎ Éditer	📄 Copier	🗑 Supprimer	5	Moreau	2501.00
☐	✎ Éditer	📄 Copier	🗑 Supprimer	6	Simon	2101.00
☐	✎ Éditer	📄 Copier	🗑 Supprimer	7	Laurent	1701.00

phpMyAdmin fonctionne en **autocommit** : chaque instruction est validée tout de suite, donc le ROLLBACK n'a rien à annuler. On a remis les salaires comme avant :

✓ 3 lignes affectées. (traitement en 0,0004 seconde(s).)

```
UPDATE employes SET salaire = salaire - 1 WHERE dept_id = 20;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

Et on a ajouté **SET autocommit = 0** au début du script :

```
SET sql_mode = ORACLE;
SET autocommit = 0;
TRUNCATE TABLE sortie;

DELIMITER $$
BEGIN
    UPDATE employes SET salaire = salaire WHERE dept_id = 20;
    afficher('Lignes : ' || SQL%ROWCOUNT);
    ROLLBACK;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

Avec ça le ROLLBACK fonctionne : toujours 3 lignes avec salaire + 1, et les salaires n'ont pas bougé :

✓ Affichage des lignes 0 - 2 (total de 3, traitement en 0,0003 seconde(s).)

```
SELECT emp_id, nom, salaire FROM employes WHERE dept_id = 20;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette t

Options supplémentaires

				emp_id	nom	salaire
☐	Éditer	Copier	Supprimer	5	Moreau	2500.00
☐	Éditer	Copier	Supprimer	6	Simon	2100.00
☐	Éditer	Copier	Supprimer	7	Laurent	1700.00

1b. Recommencez avec dept_id = 99. Que vaut SQL%ROWCOUNT ? Y a-t-il une erreur ?

Prédiction : 0 et pas d'erreur, le département 99 n'existe pas. C'est bien ça :

The screenshot shows a SQL query execution interface. At the top, a status bar indicates "Affichage des lignes 0 - 0 (total de 1, traitement en 0,0002 seconde(s))." Below this, the SQL query is displayed: `SELECT ligne FROM sortie ORDER BY num;`. The interface includes a "Profilage" checkbox and a row of links: "[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]". A control bar shows "Tout afficher" checkbox, "Nombre de lignes : 25" dropdown, and "Filtrer les lignes: Chercher dans cette table" input. A blue button labeled "Options supplémentaires" is present. At the bottom, a table header shows "ligne" and a row of actions: "Éditer", "Copier", "Supprimer", and "Lignes : 0". The "Lignes : 0" text is highlighted with a red box.

1c. Tapez maintenant le SELECT INTO, puis remplacez 99 par 10 :

```
SET sql_mode = ORACLE;

DELIMITER $$
DECLARE v_nom employees.nom%TYPE;
BEGIN
    SELECT nom INTO v_nom FROM employees WHERE dept_id = 99;
END;
$$
DELIMITER ;
```

Prédiction : une erreur "aucune donnée" avec 99 et une erreur "trop de lignes" avec 10.

Avec **99**, pas d'erreur sur MariaDB, juste un **Warning #1329 No data** (sur Oracle ce serait l'erreur ORA-01403 NO_DATA_FOUND) :

```
✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0001 seconde(s).)
```

```
SET sql_mode = ORACLE;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

```
✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0003 seconde(s).)
```

```
DECLARE v_nom employees.nom%TYPE; BEGIN SELECT nom INTO v_nom FROM employees WHERE dept_id = 99; END;;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

```
⚠ Warning: #1329 No data - zero rows fetched, selected, or processed
```

Avec **10**, cette fois c'est une vraie erreur : **#1172** (l'équivalent de ORA-01422 TOO_MANY_ROWS) :

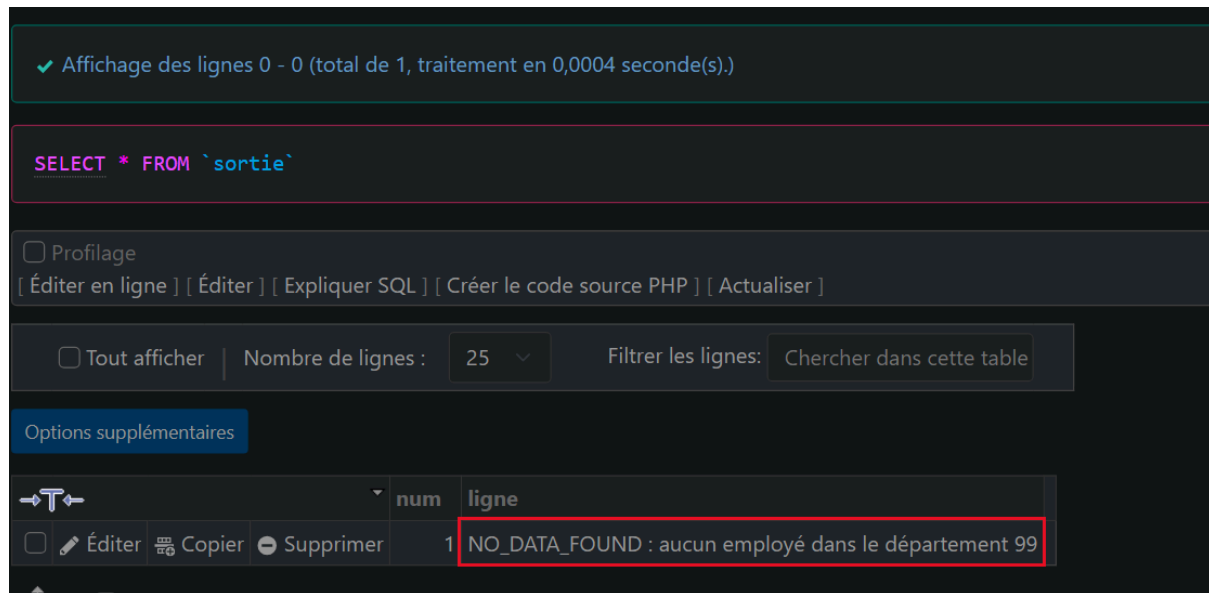
```
Requête SQL : Copier ⓘ
```

```
DECLARE v_nom employees.nom%TYPE; BEGIN SELECT nom INTO v_nom FROM employees WHERE dept_id = 10; END;;
```

MySQL a répondu : ⓘ

```
#1172 - Le résultat contient plus d'un enregistrement
```

L'exception **NO_DATA_FOUND** existe quand même sur MariaDB, on peut l'intercepter dans un bloc EXCEPTION :



À conclure :

- Un **UPDATE** qui ne touche aucune ligne n'est pas une erreur, SQL%ROWCOUNT vaut juste 0.
- Un **SELECT INTO** doit ramener exactement une ligne : 0 ligne -> NO_DATA_FOUND (sur MariaDB juste un warning si on ne l'intercepte pas), plusieurs lignes -> TOO_MANY_ROWS.
- Dès qu'une requête peut renvoyer plusieurs lignes, SELECT INTO pose problème car une variable ne peut contenir qu'une ligne. Il faut un **curseur explicite** qui lit les lignes une par une.

Exercice 2 : Curseur explicite

Le curseur liste nom et salaire des employés du département 30, avec les 4 étapes (déclarer, ouvrir, lire, fermer) et un seul FETCH. La fonction **bool_txt** sert à afficher TRUE / FALSE / NULL parce que MariaDB renvoie 1 / 0 :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
CREATE OR REPLACE FUNCTION bool_txt(p_val NUMBER) RETURN VARCHAR2 AS
BEGIN
    IF p_val IS NULL THEN
        RETURN 'NULL';
    ELSIF p_val = 1 THEN
        RETURN 'TRUE';
    ELSE
        RETURN 'FALSE';
    END IF;
END;
$$

DECLARE
    CURSOR c_emp IS SELECT nom, salaire FROM employees WHERE dept_id = 30;
    v_nom          employees.nom%TYPE;
    v_salaire      employees.salaire%TYPE;
BEGIN
    afficher('Avant OPEN      : ISOPEN=' || bool_txt(c_emp%ISOPEN));

    OPEN c_emp;
    afficher('Après OPEN      : ISOPEN=' || bool_txt(c_emp%ISOPEN)
    || ' FOUND=' || bool_txt(c_emp%FOUND)
    || ' NOTFOUND=' || bool_txt(c_emp%NOTFOUND)
    || ' ROWCOUNT=' || c_emp%ROWCOUNT);

    FETCH c_emp INTO v_nom, v_salaire;
    afficher('Ligne lue : ' || v_nom || ' - ' || v_salaire || ' €');
    afficher('Après 1er FETCH : ISOPEN=' || bool_txt(c_emp%ISOPEN)
    || ' FOUND=' || bool_txt(c_emp%FOUND)
    || ' NOTFOUND=' || bool_txt(c_emp%NOTFOUND)
    || ' ROWCOUNT=' || c_emp%ROWCOUNT);

    CLOSE c_emp;
    afficher('Après CLOSE    : ISOPEN=' || bool_txt(c_emp%ISOPEN));
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

Prédiction (de mémoire, avant de tester) :

Moment	ISOPEN	FOUND	NOTFOUND	ROWCOUNT
--------	--------	-------	----------	----------

Avant OPEN	FALSE	NULL	NULL	0
Après OPEN	TRUE	NULL	NULL	0
Après 1er FETCH	TRUE	TRUE	FALSE	1
Après CLOSE	FALSE	NULL	NULL	1

Au premier essai on affichait les 4 attributs avant le OPEN, et le bloc plante dès le début :

```

Requête SQL : Copier ⓘ

DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30; v_nom employes.nom%TYPE; v_sal
OPEN : ISOPEN=' || bool_txt(c_emp%ISOPEN) || ' FOUND=' || bool_txt(c_emp%FOUND) || ' NOTFOUND=' || bool_txt(c_emp%
c_emp%ROWCOUNT); OPEN c_emp; afficher('Après OPEN : ISOPEN=' || bool_txt(c_emp%ISOPEN) || ' FOUND=' || bool_txt(c
bool_txt(c_emp%NOTFOUND) || ' ROWCOUNT=' || c_emp%ROWCOUNT); FETCH c_emp INTO v_nom, v_salaire; afficher('Lign
1er FETCH : ISOPEN=' || bool_txt(c_emp%ISOPEN) || ' FOUND=' || bool_txt(c_emp%FOUND) || ' NOTFOUND=' || bool_txt(c_e
c_emp%ROWCOUNT); CLOSE c_emp; afficher('Après CLOSE : ISOPEN=' || bool_txt(c_emp%ISOPEN)); END;;

MySQL a répondu : ⓘ

#1326 - Cursor is not open

```

On ne peut lire FOUND, NOTFOUND et ROWCOUNT que si le curseur est ouvert.
On a testé ROWCOUNT après le CLOSE, pareil :

```

Requête SQL : Copier ⓘ

DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30; v_nom employes.nom
FETCH c_emp INTO v_nom, v_salaire; CLOSE c_emp; afficher('ROWCOUNT après CLOSE : ' || c_emp%ROWCOU

MySQL a répondu : ⓘ

#1326 - Cursor is not open

```

Donc avant OPEN et après CLOSE on affiche seulement ISOPEN. Les lignes étaient coupées dans phpMyAdmin, on a mis “Textes complets” dans “Options supplémentaires” :

✓ Affichage des lignes 0 - 4 (total de 5, traitement en 0,0003 seconde(s).)

```
SELECT * FROM `sortie`
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

	num	ligne
☐ Éditer ☐ Copier ☐ Supprimer	1	Avant OPEN : ISOPEN=FALSE
☐ Éditer ☐ Copier ☐ Supprimer	2	Après OPEN : ISOPEN=TRUE FOUND=NULL NOTFOUND=NULL ROWCOUNT=0
☐ Éditer ☐ Copier ☐ Supprimer	3	Ligne lue : Garcia - 4100.00 €
☐ Éditer ☐ Copier ☐ Supprimer	4	Après 1er FETCH : ISOPEN=TRUE FOUND=TRUE NOTFOUND=FALSE ROWCOUNT=1
☐ Éditer ☐ Copier ☐ Supprimer	5	Après CLOSE : ISOPEN=FALSE

Mesures :

Moment	ISOPEN	FOUND	NOTFOUND	ROWCOUNT
Avant OPEN	FALSE	erreur #1326	erreur #1326	erreur #1326
Après OPEN	TRUE	NULL	NULL	0
Après 1er FETCH	TRUE	TRUE	FALSE	1
Après CLOSE	FALSE	erreur #1326	erreur #1326	erreur #1326

À conclure : ce qui nous a surpris c’est FOUND, NOTFOUND et ROWCOUNT avant le OPEN et après le CLOSE : on pensait avoir NULL ou 0, mais c’est une erreur (#1326 Cursor is not open, sur Oracle c’est INVALID_CURSOR). Juste après le OPEN, FOUND et NOTFOUND sont NULL car aucun FETCH n’a encore été fait. Pour %ROWCOUNT après CLOSE, il ne garde pas sa valeur : on ne peut plus le lire du tout, il faut le récupérer avant de fermer le curseur.

Exercice 3 : Casser le curseur

On part du code de l'exercice 2 et on fait chaque manipulation dans un bloc à part (fichier 13_ex3_manipulations.sql). Nos prédictions et les résultats :

Manipulation	Prédiction	Résultat réel
FETCH avant le OPEN	erreur curseur pas ouvert (ORA-01001)	#1326 Cursor is not open
Deux OPEN de suite	erreur curseur déjà ouvert (ORA-06511)	#1325 Cursor is already open
Deux CLOSE de suite	pas d'erreur, il est déjà fermé	#1326 Cursor is not open
FETCH dans une variable de mauvais type	erreur de type (ORA-06502)	pas d'erreur, Warning #1366 et la variable vaut 0
FETCH INTO avec une seule variable pour 2 colonnes	erreur sur le nombre de variables	#1328 Incorrect number of FETCH variables

FETCH avant le OPEN :

```
Requête SQL : Copier ⓘ  
  
DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30; v_nom employes.nom%TYPE; v_salaire employes.salaire%TYPE;  
INTO v_nom, v_salaire; OPEN c_emp; CLOSE c_emp; END;;  
  
MySQL a répondu : ⓘ  
  
#1326 - Cursor is not open
```

Deux OPEN de suite :

```
Requête SQL : Copier ⓘ  
  
DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30; BEGIN OPEN c_emp; OPEN c_emp; CLOSE c_emp; END;;  
  
MySQL a répondu : ⓘ  
  
#1325 - Cursor is already open
```

Deux CLOSE de suite, on pensait que ça passerait mais non :

Requête SQL : [Copier](#) ⓘ

```
DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30; BEGIN OPEN c_emp; CLOS
```

MySQL a répondu : ⓘ

```
#1326 - Cursor is not open
```

FETCH dans une variable **NUMBER** pour le nom :

```
DELIMITER $$
DECLARE
  CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30;
  v_nom          NUMBER;
  v_salaire employees.salaire%TYPE;
BEGIN
  OPEN c_emp;
  FETCH c_emp INTO v_nom, v_salaire;
  afficher('Nom lu : ' || v_nom || ' / salaire : ' || v_salaire);
  CLOSE c_emp;
END;
$$
DELIMITER ;
```

Pas d'erreur ! MariaDB convertit 'Garcia' en nombre et met 0 dans la variable, avec juste un warning :

✓ Affichage des lignes 0 - 0 (total de 1, traitement en 0,0002 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

ligne
Nom lu : 0 / salaire : 4100.00

FETCH INTO avec une seule variable pour nom et salaire :

Requête SQL : Copier ?

```
DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30; v_nom employes.nom%TYPE;  
c_emp; END;;
```

MySQL a répondu : ?

#1328 - Incorrect number of FETCH variables

Défi

On a d'abord mis **WHEN CURSOR_ALREADY_OPEN** comme dans le TP, mais MariaDB ne la connaît pas :

Requête SQL : Copier ?

```
DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30; BEGIN OPEN c_emp; OPEN  
afficher('Erreur 1 : le curseur est déjà ouvert !'); END;;
```

MySQL a répondu : ?

#1319 - Undefined CONDITION: cursor_already_open

On la déclare nous même avec **CONDITION FOR 1325** (le code de l'erreur des deux OPEN). INVALID_CURSOR, elle, existe bien :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
    CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30;
    cursor_already_open CONDITION FOR 1325;
BEGIN
    OPEN c_emp;
    OPEN c_emp;
EXCEPTION
    WHEN cursor_already_open THEN
        afficher('Erreur 1 : le curseur est déjà ouvert !');
END;
$$

DECLARE
    CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30;
    v_nom      employes.nom%TYPE;
    v_salaire  employes.salaire%TYPE;
BEGIN
    FETCH c_emp INTO v_nom, v_salaire;
EXCEPTION
    WHEN INVALID_CURSOR THEN
        afficher('Erreur 2 : on ne peut pas lire un curseur qui n''est pas ouvert !');
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

Les deux erreurs sont interceptées et on affiche notre message :

✓ Affichage des lignes 0 - 1 (total de 2, traitement en 0,0002 seconde(s).)

```
SELECT * FROM `sortie`
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Au

Options supplémentaires

			num	ligne
☐	Éditer	Copier	Supprimer	1
				Erreur 1 : le curseur est déjà ouvert !
☐	Éditer	Copier	Supprimer	2
				Erreur 2 : on ne peut pas lire un curseur qui n'est pas ouvert !

Exercice 4 : Trois façons de boucler

4a. Avec LOOP ... EXIT WHEN :

Le EXIT WHEN se met juste après le FETCH :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30 ORDER BY emp_id;
  v_nom      employes.nom%TYPE;
  v_salaire  employes.salaire%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_nom, v_salaire;
    EXIT WHEN c_emp%NOTFOUND;
    afficher(c_emp%ROWCOUNT || '. ' || v_nom || ' : ' || v_salaire || ' €');
  END LOOP;
  CLOSE c_emp;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

✓ Affichage des lignes 0 - 4 (total de 5, traitement en 0,0002 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
 [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table Trier par clé

Options supplémentaires

				ligne
☐	Éditer	Copier	Supprimer	1. Garcia : 4100.00 €
☐	Éditer	Copier	Supprimer	2. Roux : 3600.00 €
☐	Éditer	Copier	Supprimer	3. Fournier : 3000.00 €
☐	Éditer	Copier	Supprimer	4. Girard : 2200.00 €
☐	Éditer	Copier	Supprimer	5. Bonnet : 1800.00 €

4b. Avec WHILE :

Il faut un premier FETCH avant la boucle pour que %FOUND soit TRUE, puis un FETCH à la fin de chaque tour :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30 ORDER BY emp_id;
  v_nom      employes.nom%TYPE;
  v_salaire  employes.salaire%TYPE;
BEGIN
  OPEN c_emp;
  FETCH c_emp INTO v_nom, v_salaire;
  WHILE c_emp%FOUND LOOP
    afficher(c_emp%ROWCOUNT || '. ' || v_nom || ' : ' || v_salaire || ' €');
    FETCH c_emp INTO v_nom, v_salaire;
  END LOOP;
  CLOSE c_emp;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

✓ Affichage des lignes 0 - 4 (total de 5, traitement en 0,0002 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
 [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

				ligne
☐	✎ Éditer	📄 Copier	🗑 Supprimer	1. Garcia : 4100.00 €
☐	✎ Éditer	📄 Copier	🗑 Supprimer	2. Roux : 3600.00 €
☐	✎ Éditer	📄 Copier	🗑 Supprimer	3. Fournier : 3000.00 €
☐	✎ Éditer	📄 Copier	🗑 Supprimer	4. Girard : 2200.00 €
☐	✎ Éditer	📄 Copier	🗑 Supprimer	5. Bonnet : 1800.00 €

4c. Avec FOR rec IN curseur LOOP :

Pas de OPEN, FETCH ni CLOSE, le curseur s'ouvre et se ferme tout seul et **rec** contient la ligne :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30 ORDER BY emp_id;
BEGIN
  FOR rec IN c_emp LOOP
    afficher(rec.nom || ' : ' || rec.salaire || ' €');
  END LOOP;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

✓ Affichage des lignes 0 - 4 (total de 5, traitement en 0,0002 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
[\[Éditer en ligne \]](#) [\[Éditer \]](#) [\[Expliquer SQL \]](#) [\[Créer le code source PHP \]](#) [\[Actualiser \]](#)

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Trier par clé :

Options supplémentaires

				ligne
☐	Éditer	Copier	Supprimer	Garcia : 4100.00 €
☐	Éditer	Copier	Supprimer	Roux : 3600.00 €
☐	Éditer	Copier	Supprimer	Fournier : 3000.00 €
☐	Éditer	Copier	Supprimer	Girard : 2200.00 €
☐	Éditer	Copier	Supprimer	Bonnet : 1800.00 €

4d. Expérience : EXIT WHEN c%NOTFOUND déplacé après l’affichage :

```
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_nom, v_salaire;
    afficher(c_emp%ROWCOUNT || ' ' || v_nom || ' : ' || v_salaire || ' €');
    EXIT WHEN c_emp%NOTFOUND;
  END LOOP;
  CLOSE c_emp;
END;
```

Prédiction : la dernière ligne va s’afficher deux fois, car le dernier FETCH ne trouve rien mais les variables gardent l’ancienne valeur.

C'est bien ça, Bonnet apparait deux fois (et %ROWCOUNT reste à 5) :

✓ Affichage des lignes 0 - 5 (total de 6, traitement en 0,0002 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table Trier p

Options supplémentaires

				ligne
☐	Éditer	Copier	Supprimer	1. Garcia : 4100.00 €
☐	Éditer	Copier	Supprimer	2. Roux : 3600.00 €
☐	Éditer	Copier	Supprimer	3. Fournier : 3000.00 €
☐	Éditer	Copier	Supprimer	4. Girard : 2200.00 €
☐	Éditer	Copier	Supprimer	5. Bonnet : 1800.00 €
☐	Éditer	Copier	Supprimer	5. Bonnet : 1800.00 €

4e. Expérience : oublier un des deux FETCH du WHILE :

Sans le premier FETCH avant le WHILE, prédiction : rien ne s'affiche, %FOUND vaut NULL au départ donc la condition du WHILE n'est pas vraie.

Résultat : la table sortie est vide et il n'y a pas d'erreur, donc c'est un bug qui passe inaperçu :

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0002 seconde(s).)

```
SET sql_mode = ORACLE;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0003 seconde(s).)

```
TRUNCATE TABLE sortie;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0003 seconde(s).)

```
DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30 ORDER BY emp_id; v_nom employ  
employes.salaire%TYPE; BEGIN OPEN c_emp; WHILE c_emp%FOUND LOOP afficher(c_emp%ROWCOUNT || ' ' || v_nom |  
FETCH c_emp INTO v_nom, v_salaire; END LOOP; CLOSE c_emp; END;;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0009 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage

[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

ligne

Opérations sur les résultats de la requête

Conserver cette requête SQL dans les signets

Intitulé :

☐ Signet visible pour les autres utilisateurs

Conserver cette requête SQL dans les signets

Sans le FETCH dans la boucle, prédiction : boucle infinie, %FOUND reste toujours TRUE :

```
BEGIN
  OPEN c_emp;
  FETCH c_emp INTO v_nom, v_salaire;
  WHILE c_emp%FOUND LOOP
    | afficher(c_emp%ROWCOUNT || '. ' || v_nom || ' : ' || v_salaire || ' €');
  END LOOP;
  CLOSE c_emp;
END;
```

La requête ne s'arrête jamais. Pour l'interrompre, on va dans un autre onglet dans "État" > "Processus". La requête qui tourne est l'INSERT de la procédure afficher(), appelée à chaque tour :

Processus	ID	Utilisateur	Hôte	Base de données	Commande	Temps	État	Progression	Requête SQL ↕
Tuer	1	system user		Aucun(e)	Daemon		InnoDB purge coordinator	0.000	---
Tuer	2	system user		Aucun(e)	Daemon		InnoDB purge worker	0.000	---
Tuer	3	system user		Aucun(e)	Daemon		InnoDB purge worker	0.000	---
Tuer	4	system user		Aucun(e)	Daemon		InnoDB purge worker	0.000	---
Tuer	5	system user		Aucun(e)	Daemon		InnoDB shutdown handler	0.000	---
Tuer	714	pma	localhost:63915	phpmyadmin	Sleep	4	---	0.000	---
Tuer	715	root	localhost:63916	tp_curseurs	Query	4	Closing tables	0.000	INSERT INTO sortie(ligne) VALUES (NAME_CONST('p_texte', _utf8mb4'1. Garcia : 4100.00 €' COLLATE 'u
Tuer	718	pma	localhost:56605	phpmyadmin	Sleep	0	---	0.000	---
Tuer	719	root	localhost:56606	mysql	Query	0	Init	0.000	SHOW PROCESSLIST

On clique sur “Tuer” sur cette ligne :

Processus

Filtres

☐ N'afficher que les valeurs actives Actua

✓ Le processus 715 a été éliminé.

Processus	ID	Utilisateur	Hôte	Base de données	Commande	Temps	État	Progression	Requête SQL
Tuer	1	system user		Aucun(e)	Daemon		InnoDB purge coordinator	0.000	---
Tuer	2	system user		Aucun(e)	Daemon		InnoDB purge worker	0.000	---
Tuer	3	system user		Aucun(e)	Daemon		InnoDB purge worker	0.000	---
Tuer	4	system user		Aucun(e)	Daemon		InnoDB purge worker	0.000	---
Tuer	5	system user		Aucun(e)	Daemon		InnoDB shutdown handler	0.000	---
Tuer	714	pma	localhost:63915	phpmyadmin	Sleep	4	---	0.000	---
Tuer	718	pma	localhost:56605	phpmyadmin	Sleep	0	---	0.000	---
Tuer	719	root	localhost:56606	mysql	Query	0	Init	0.000	SHOW PROCESSLIST

⚠ NB : activer l'actualisation automatique peut causer un trafic important entre le serveur Web et le serveur MySQL.

Fréquence de rafraîchissement:

5 secondes

► Lancer l'actualisation automatique

Dans le premier onglet on obtient l'erreur #2006 MySQL server has gone away :

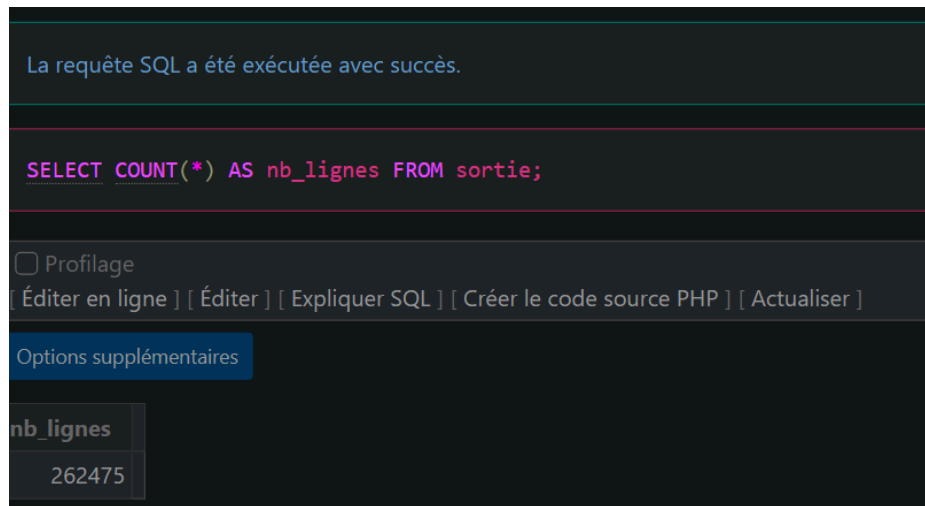
Requête SQL : Copier ?

```
DECLARE CURSOR c_emp IS SELECT nom, salaire FROM employes WHERE dept_id = 30 ORDER BY emp_id;
OPEN c_emp;
FETCH c_emp INTO v_nom, v_salaire;
WHILE c_emp%FOUND LOOP
  afficher(c_emp%ROWCOUNT);
END;;
```

MySQL a répondu : ?

#2006 - MySQL server has gone away

En une dizaine de secondes, la boucle a écrit **262 475 lignes** dans la table sortie (on l’a vidée après) :



La première fois on cherchait la ligne “DECLARE...” dans la liste des processus alors que c’est l’INSERT qui s’affiche, le temps de trouver la table avait plus de 10 millions de lignes.

À conclure :

- **LOOP / EXIT WHEN** : le plus libre (on peut sortir quand on veut), mais il faut bien mettre le EXIT juste après le FETCH sinon la dernière ligne est en double.
- **WHILE** : lisible, mais il faut deux FETCH. Si on oublie le premier il ne se passe rien, si on oublie celui de la boucle c’est une boucle infinie.
- **FOR** : le plus court et le plus sûr, pas de OPEN / FETCH / CLOSE à gérer donc pas d’oubli possible, et le curseur est refermé tout seul même en cas d’erreur. C’est celui à préférer pour parcourir toutes les lignes. LOOP et WHILE servent quand on veut contrôler la lecture (s’arrêter avant la fin, lire une seule ligne, utiliser %ROWCOUNT avant le CLOSE).

Exercice 5 : Curseurs paramétrés

5a. Curseur c_emp (p_dept NUMBER, p_seuil NUMBER) appelé avec (30, 3000) puis (10, 2000) :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_emp(p_dept NUMBER, p_seuil NUMBER) IS
    SELECT nom, salaire FROM employes
    WHERE dept_id = p_dept AND salaire > p_seuil
    ORDER BY salaire DESC;
BEGIN
  afficher('--- Département 30, salaire > 3000 ---');
  FOR rec IN c_emp(30, 3000) LOOP
    afficher(rec.nom || ' : ' || rec.salaire || ' €');
  END LOOP;

  afficher('--- Département 10, salaire > 2000 ---');
  FOR rec IN c_emp(10, 2000) LOOP
    afficher(rec.nom || ' : ' || rec.salaire || ' €');
  END LOOP;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

Prédiction : Garcia et Roux pour le 30 (Fournier est à 3000 pile, il ne “dépasse” pas), Martin, Durand et Petit pour le 10.

✓ Affichage des lignes 0 - 6 (total de 7, traitement en 0,0002 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage

[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher

Nombre de lignes :

25

Filtrer les lignes:

Chercher dans cette table

Trier par c

Options supplémentaires

				ligne
☐	Éditer	Copier	Supprimer	--- Département 30, salaire > 3000 ---
☐	Éditer	Copier	Supprimer	Garcia : 4100.00 €
☐	Éditer	Copier	Supprimer	Roux : 3600.00 €
☐	Éditer	Copier	Supprimer	--- Département 10, salaire > 2000 ---
☐	Éditer	Copier	Supprimer	Martin : 3200.00 €
☐	Éditer	Copier	Supprimer	Durand : 2800.00 €
☐	Éditer	Copier	Supprimer	Petit : 2800.00 €

5b. Boucle externe sur departements, avec un seuil de 2500 et un en-tête par département :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
    CURSOR c_dept IS SELECT dept_id, dept_nom FROM departements ORDER BY dept_id;
    CURSOR c_emp(p_dept NUMBER, p_seuil NUMBER) IS
        SELECT nom, salaire FROM employes
        WHERE dept_id = p_dept AND salaire > p_seuil
        ORDER BY salaire DESC;
BEGIN
    FOR d IN c_dept LOOP
        afficher('== ' || d.dept_id || ' - ' || d.dept_nom || ' ==');
        FOR e IN c_emp(d.dept_id, 2500) LOOP
            afficher('- ' || e.nom || ' : ' || e.salaire || ' €');
        END LOOP;
    END LOOP;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

Le département 20 n'affiche personne (Moreau est à 2500 pile) et le 40 n'a aucun employé :

✓ Affichage des lignes 0 - 9 (total de 10, traitement en 0,0002 seconde(s))

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé :

Options supplémentaires

				ligne
☐	Éditer	Copier	Supprimer	=== 10 - Compta ===
☐	Éditer	Copier	Supprimer	- Martin : 3200.00 €
☐	Éditer	Copier	Supprimer	- Durand : 2800.00 €
☐	Éditer	Copier	Supprimer	- Petit : 2800.00 €
☐	Éditer	Copier	Supprimer	=== 20 - RH ===
☐	Éditer	Copier	Supprimer	=== 30 - IT ===
☐	Éditer	Copier	Supprimer	- Garcia : 4100.00 €
☐	Éditer	Copier	Supprimer	- Roux : 3600.00 €
☐	Éditer	Copier	Supprimer	- Fournier : 3000.00 €
☐	Éditer	Copier	Supprimer	=== 40 - Archives ===

5c. Afficher “aucun employé” sans faire de COUNT(*) séparé :

Premier essai avec %ROWCOUNT après la boucle FOR interne :

```
BEGIN
  FOR d IN c_dept LOOP
    afficher('== ' || d.dept_id || ' - ' || d.dept_nom || ' ==');
    FOR e IN c_emp(d.dept_id, 2500) LOOP
      afficher('- ' || e.nom || ' : ' || e.salaire || ' €');
    END LOOP;
    IF c_emp%ROWCOUNT = 0 THEN
      afficher('aucun employé');
    END IF;
  END LOOP;
END;
```

Erreur #1326 : à la sortie d’une boucle FOR le curseur est déjà fermé, on ne peut plus lire %ROWCOUNT (comme dans l’exercice 2) :

Requête SQL : [Copier](#) ⓘ

```
DECLARE CURSOR c_dept IS SELECT dept_id, dept_nom FROM departements ORDER BY dept_id; CURSOR c_emp(p_dept NUMBER) IS
  SELECT emp_id, nom, salaire FROM employes WHERE dept_id = p_dept AND salaire > p_seuil ORDER BY salaire DESC;
BEGIN FOR d IN c_dept LOOP afficher('== ' || d.dept_id || ' - ' || d.dept_nom || ' ==');
  FOR e IN c_emp(d.dept_id, 2500) LOOP afficher('- ' || e.nom || ' : ' || e.salaire || ' €'); END LOOP;
  IF c_emp%ROWCOUNT = 0 THEN afficher('aucun employé'); END IF; END LOOP; END;
```

MySQL a répondu : ⓘ

#1326 - Cursor is not open

On utilise donc un compteur **v_nb** remis à 0 pour chaque département et augmenté dans la boucle (un indicateur booléen aurait marché aussi). Si il vaut 0 après la boucle, on affiche “aucun employé”.

5d. Défi : salaire moyen des employés affichés, sans division par zéro :

On garde aussi le total des salaires, et on ne fait la division que dans le ELSE, donc quand `v_nb > 0` :

```
SET sql_mode = ORACLE;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
    CURSOR c_dept IS SELECT dept_id, dept_nom FROM departements ORDER BY dept_id;
    CURSOR c_emp(p_dept NUMBER, p_seuil NUMBER) IS
        SELECT nom, salaire FROM employes
        WHERE dept_id = p_dept AND salaire > p_seuil
        ORDER BY salaire DESC;
    v_nb    NUMBER;
    v_total NUMBER;
BEGIN
    FOR d IN c_dept LOOP
        afficher('=== ' || d.dept_id || ' - ' || d.dept_nom || ' ===');
        v_nb := 0;
        v_total := 0;
        FOR e IN c_emp(d.dept_id, 2500) LOOP
            afficher('- ' || e.nom || ' : ' || e.salaire || ' €');
            v_nb := v_nb + 1;
            v_total := v_total + e.salaire;
        END LOOP;

        IF v_nb = 0 THEN
            afficher('aucun employé');
        ELSE
            afficher('Salaire moyen : ' || ROUND(v_total / v_nb, 2) || ' €');
        END IF;
    END LOOP;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

✓ Affichage des lignes 0 - 13 (total de 14, traitement en 0,0004 seconde(s).)

```
SELECT ligne FROM sortie ORDER BY num;
```

☐ Profilage
 [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table Trier par cl

Options supplémentaires

				ligne
☐	Éditer	Copier	Supprimer	=== 10 - Compta ===
☐	Éditer	Copier	Supprimer	- Martin : 3200.00 €
☐	Éditer	Copier	Supprimer	- Durand : 2800.00 €
☐	Éditer	Copier	Supprimer	- Petit : 2800.00 €
☐	Éditer	Copier	Supprimer	Salaire moyen : 2933.33 €
☐	Éditer	Copier	Supprimer	=== 20 - RH ===
☐	Éditer	Copier	Supprimer	aucun employé
☐	Éditer	Copier	Supprimer	=== 30 - IT ===
☐	Éditer	Copier	Supprimer	- Garcia : 4100.00 €
☐	Éditer	Copier	Supprimer	- Roux : 3600.00 €
☐	Éditer	Copier	Supprimer	- Fournier : 3000.00 €
☐	Éditer	Copier	Supprimer	Salaire moyen : 3566.67 €
☐	Éditer	Copier	Supprimer	=== 40 - Archives ===
☐	Éditer	Copier	Supprimer	aucun employé

Le département 20 affiche aussi “aucun employé” : avec le seuil de 2500 personne n’est affiché, c’est logique vu que le compteur ne compte que les employés affichés.

Exercice 6 : FOR UPDATE

Pour avoir deux sessions, on ouvre phpMyAdmin deux fois : la **Session 1** dans un onglet normal et la **Session 2** dans une fenêtre de navigation privée (sinon les deux onglets partagent la même session PHP). Attention, avec phpMyAdmin chaque clic sur “Exécuter” ouvre une connexion à MariaDB qui se ferme à la fin du script.

6a. Session 1 : primes de 5 % pour le département 10 avec FOR UPDATE et WHERE CURRENT OF, sans COMMIT :

WHERE CURRENT OF n'existe pas sur MariaDB :

Requête SQL : [Copier](#) ?

```
DECLARE CURSOR c_prime IS SELECT emp_id, nom, salaire FROM employees WHERE dept_id = 10 AND prime IS NULL;
OPEN c_prime;
LOOP
  FETCH c_prime INTO emp_id, nom, salaire;
  IF emp_id = 0 THEN EXIT;
  UPDATE employees SET prime = r.salaire * 0.05 WHERE CURRENT OF c_prime; afficher(r.nom || ' : nouvelle prime de ' || r.salaire * 0.05);
END LOOP;
```

MySQL a répondu : ?

```
#1064 - Erreur de syntaxe près de 'OF c_prime;
      afficher(r.nom || ' : nouvelle prime de ' || r.salaire * 0.05...' à la ligne 8
```

On passe donc par la clé primaire (WHERE emp_id = r.emp_id).
DBMS_SESSION.SLEEP n'existe pas non plus, on utilise **DO SLEEP(60)**. Et il faut **SET autocommit = 0**, sinon chaque UPDATE est validé tout de suite et le verrou disparaît :

```
SET sql_mode = ORACLE;
SET autocommit = 0;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_prime IS
    SELECT emp_id, nom, salaire FROM employes
    WHERE dept_id = 10 AND prime IS NULL
    FOR UPDATE;
BEGIN
  FOR r IN c_prime LOOP
    UPDATE employes SET prime = r.salaire * 0.05 WHERE emp_id = r.emp_id;
    afficher(r.nom || ' : nouvelle prime de ' || r.salaire * 0.05 || ' €');
  END LOOP;

  DO SLEEP(60);
END;
$$
DELIMITER ;

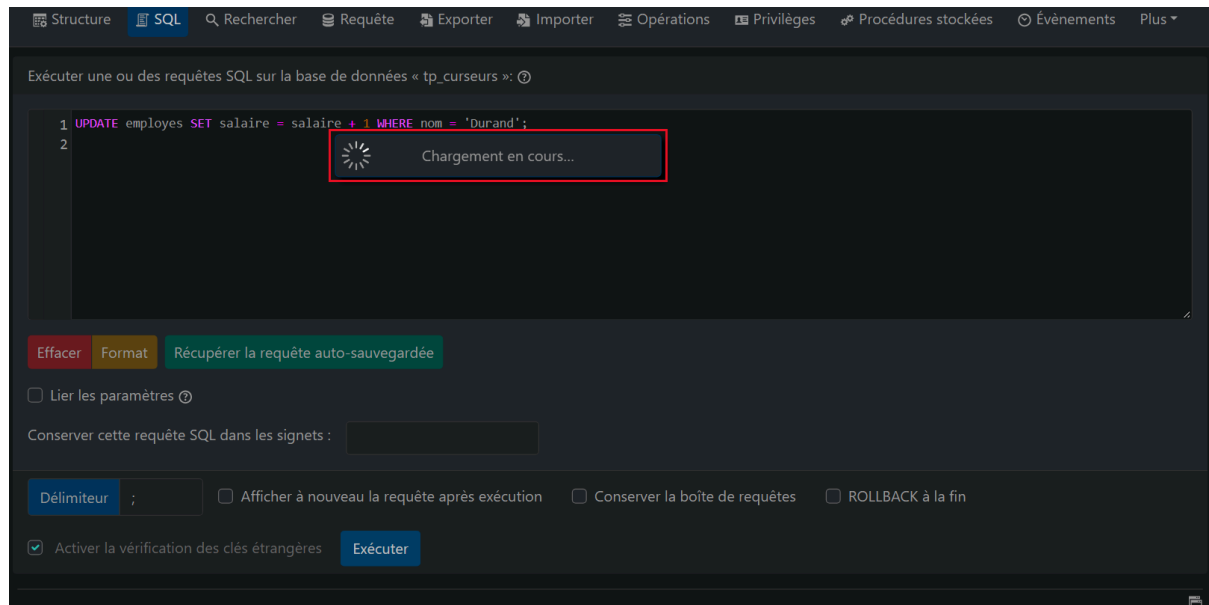
SELECT ligne FROM sortie ORDER BY num;
```

6b. Session 2, pendant les 60 secondes :

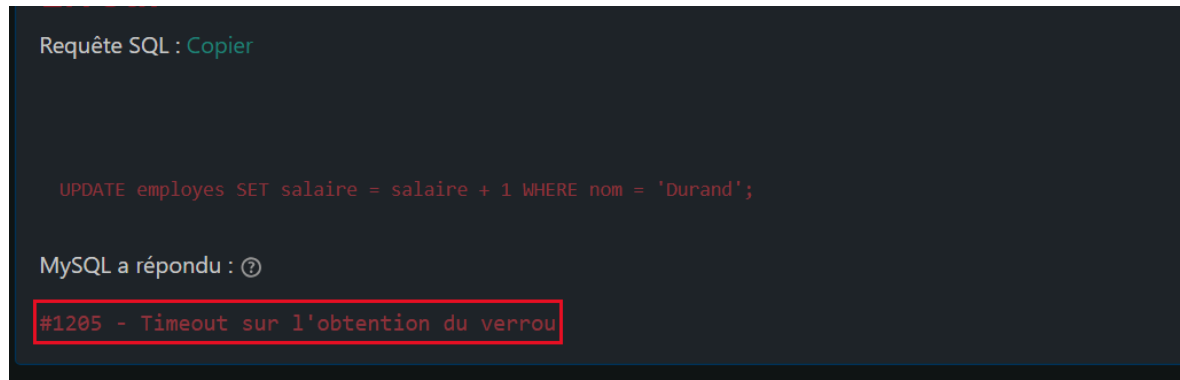
```
UPDATE employes SET salaire = salaire + 1 WHERE nom = 'Durand';
```

Prédiction : l'UPDATE va attendre que la session 1 libère la ligne de Durand.

La requête reste bloquée sur “Chargement en cours...” :



Au bout d'environ **50 secondes** (on a chronométré 52 s) elle s'arrête avec l'erreur **#1205** :



C'est le délai d'attente d'un verrou sur MariaDB (innodb_lock_wait_timeout = 50 s par défaut). Il est plus court que la pause de 60 s de la session 1, donc la session 2 abandonne avant.

Pendant ce temps la session 1 a bien calculé les primes (Durand et Leroy) :

✓ Affichage des lignes 0 - 1 (total de 2, traitement en 0,0002 seconde(s).)

```
SELECT * FROM `sortie`
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

	num	ligne
☐ Éditer ☐ Copier ☐ Supprimer	1	Durand : nouvelle prime de 140.0000 €
☐ Éditer ☐ Copier ☐ Supprimer	2	Leroy : nouvelle prime de 95.0000 €

6c. SELECT dans la Session 2 une fois le bloc de la Session 1 terminé :

✓ Affichage des lignes 0 - 3 (total de 4, traitement en 0,0003 seconde(s).)

```
SELECT emp_id, nom, salaire, prime FROM employes WHERE dept_id = 10;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

	emp_id	nom	salaire	prime
☐ Éditer ☐ Copier ☐ Supprimer	1	Martin	3200.00	500.00
☐ Éditer ☐ Copier ☐ Supprimer	2	Durand	2800.00	NULL
☐ Éditer ☐ Copier ☐ Supprimer	3	Petit	2800.00	300.00
☐ Éditer ☐ Copier ☐ Supprimer	4	Leroy	1900.00	NULL

Les primes sont toujours NULL : pas de COMMIT, et comme phpMyAdmin ferme la connexion à la fin du script, MariaDB fait un ROLLBACK automatique. On ne peut donc pas faire le COMMIT en Session 1 “après coup” comme dans SQL Developer.

On a donc mis le **COMMIT** à la fin du bloc, après une pause de 30 s :

```
SET sql_mode = ORACLE;
SET autocommit = 0;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_prime IS
    SELECT emp_id, nom, salaire FROM employes
    WHERE dept_id = 10 AND prime IS NULL
    FOR UPDATE;
BEGIN
  FOR r IN c_prime LOOP
    UPDATE employes SET prime = r.salaire * 0.05 WHERE emp_id = r.emp_id;
    afficher(r.nom || ' : nouvelle prime de ' || r.salaire * 0.05 || ' €');
  END LOOP;

  DO SLEEP(30);
  COMMIT;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

Pendant la pause, le SELECT de la session 2 passe sans attendre (un SELECT simple ne pose pas de verrou) mais il ne voit pas encore les primes :

✓ Affichage des lignes 0 - 3 (total de 4, traitement en 0,0004 seconde(s).)

```
SELECT emp_id, nom, salaire, prime FROM employes WHERE dept_id = 10;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table Trier p

Options supplémentaires

				emp_id	nom	salaire	prime
☐	Éditer	Copier	Supprimer	1	Martin	3200.00	500.00
☐	Éditer	Copier	Supprimer	2	Durand	2800.00	NULL
☐	Éditer	Copier	Supprimer	3	Petit	2800.00	300.00
☐	Éditer	Copier	Supprimer	4	Leroy	1900.00	NULL

↑ ☐ Tout cacher Avec la sélection Éditer Copier Supprimer Exporter

Après le COMMIT, la session 2 voit les nouvelles primes :

✓ Affichage des lignes 0 - 3 (total de 4, traitement en 0,0003 seconde(s))

```
SELECT emp_id, nom, salaire, prime FROM employes WHERE dept_id = 10;
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table Trier par

Options supplémentaires

				emp_id	nom	salaire	prime
☐	Éditer	Copier	Supprimer	1	Martin	3200.00	500.00
☐	Éditer	Copier	Supprimer	2	Durand	2800.00	140.00
☐	Éditer	Copier	Supprimer	3	Petit	2800.00	300.00
☐	Éditer	Copier	Supprimer	4	Leroy	1900.00	95.00

Une modification n'est visible par les autres sessions qu'**après le COMMIT**. (On a remis les primes à NULL pour la suite.)

6d. FOR UPDATE NOWAIT, dans l'autre sens :

Avec **FOR UPDATE NOWAIT** dans le curseur, MariaDB refuse directement :

Requête SQL : Copier ⓘ

```
DECLARE CURSOR c_prime IS SELECT emp_id, nom, salaire FROM employes WHERE dept_id = 10 AND prime IS NULL;  
UPDATE employes SET prime = r.salaire * 0.05 WHERE emp_id = r.emp_id; afficher(r.nom || ' : nouvelle prime de ' || r.salaire * 0.05);
```

MySQL a répondu : ⓘ

```
#1314 - [NO]WAIT is not allowed in stored procedures
```

NOWAIT n'est pas autorisé dans les blocs et les procédures. On a donc fait l'inverse : la session 2 verrouille Durand (UPDATE sans COMMIT + pause de 30 s) et la session 1 fait un SELECT ... FOR UPDATE NOWAIT en dehors d'un bloc. L'erreur arrive tout de suite au lieu d'attendre 50 secondes :

Requête SQL : Copier ⓘ

```
SELECT emp_id, nom, prime FROM employees WHERE dept_id = 10 AND prime IS NULL FOR UPDATE NOWAIT;
```

MySQL a répondu : ⓘ

```
#1205 - Timeout sur l'obtention du verrou
```

Dans un bloc on a à peu près le même résultat en mettant le délai d'attente à 1 seconde et en interceptant l'erreur 1205 :

```
SET sql_mode = ORACLE;
SET autocommit = 0;
SET innodb_lock_wait_timeout = 1;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_prime IS
    SELECT emp_id, nom, salaire FROM employees
    WHERE dept_id = 10 AND prime IS NULL
    FOR UPDATE;
  ligne_verrouillee CONDITION FOR 1205;
BEGIN
  FOR r IN c_prime LOOP
    UPDATE employees SET prime = r.salaire * 0.05 WHERE emp_id = r.emp_id;
    afficher(r.nom || ' : nouvelle prime de ' || r.salaire * 0.05 || ' €');
  END LOOP;
  COMMIT;
EXCEPTION
  WHEN ligne_verrouillee THEN
    afficher('Lignes déjà verrouillées par une autre session, on réessaiera plus tard');
    ROLLBACK;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

✓ Affichage des lignes 0 - 0 (total de 1, traitement en 0,0002 seconde(s).)

```
SELECT * FROM `sortie`
```

☐ Profilage
 [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

	num	ligne
☐ Éditer ☐ Copier ☐ Supprimer	1	Lignes déjà verrouillées par une autre session, on réessaiera plus tard

Différence : sans NOWAIT la session attend que le verrou se libère (jusqu'à 50 s), avec NOWAIT elle échoue immédiatement et on peut prévenir l'utilisateur. L'exception à intercepter est **ORA-00054** (resource busy) sur Oracle, qu'on déclare avec PRAGMA EXCEPTION_INIT, et l'erreur **1205** sur MariaDB.

6e. COMMIT à l'intérieur de la boucle FOR UPDATE (département 20) :

```
SET sql_mode = ORACLE;
SET autocommit = 0;
TRUNCATE TABLE sortie;

DELIMITER $$
DECLARE
  CURSOR c_rh IS
    SELECT emp_id, nom, prime FROM employes
    WHERE dept_id = 20
    FOR UPDATE;
BEGIN
  FOR r IN c_rh LOOP
    UPDATE employes SET prime = NVL(r.prime, 0) + 50 WHERE emp_id = r.emp_id;
    COMMIT;
    afficher(r.nom || ' : prime passée à ' || (NVL(r.prime, 0) + 50) || ' € (COMMIT fait)');
  END LOOP;
END;
$$
DELIMITER ;

SELECT ligne FROM sortie ORDER BY num;
```

Prédiction : une erreur, sur Oracle c'est l'ORA-01002 "fetch out of sequence".

Sur MariaDB aucune erreur, les 3 employés sont traités :

✓ Affichage des lignes 0 - 2 (total de 3, traitement en 0,0002 seconde(s).)

SELECT ligne FROM sortie ORDER BY num;

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table Trier p

Options supplémentaires

				ligne
☐	✎ Éditer	📄 Copier	🗑 Supprimer	Moreau : prime passée à 250.00 € (COMMIT fait)
☐	✎ Éditer	📄 Copier	🗑 Supprimer	Simon : prime passée à 50.00 € (COMMIT fait)
☐	✎ Éditer	📄 Copier	🗑 Supprimer	Laurent : prime passée à 150.00 € (COMMIT fait)

MariaDB copie le résultat du curseur au moment du OPEN, donc le COMMIT ne casse pas la lecture. Par contre le premier COMMIT libère tous les verrous du FOR UPDATE, les lignes suivantes ne sont plus protégées donc ça n'a plus d'intérêt. (On a relancé le script de l'étape 0 après pour remettre les données.)

À conclure : FOR UPDATE verrouille les lignes du curseur dès le OPEN, pour que personne ne puisse les modifier entre le moment où on les lit et le moment où on les met à jour, jusqu'au COMMIT ou au ROLLBACK. **WHERE CURRENT OF** modifie exactement la ligne sur laquelle est le curseur (Oracle utilise son ROWID), alors qu'un WHERE emp_id = ... refait une recherche : si la colonne n'est pas unique ou a changé on peut modifier d'autres lignes, et c'est moins rapide.

Exercice 7 : Mini-projet

Sujet : un rapport de paie par département avec le nom du département, les employés avec salaire et prime, le total et le salaire maximum, un message pour un département vide et un total général.

On a fait une procédure **rapport_paie(p_dept)** : avec NULL elle affiche tous les départements, sinon un seul. Un curseur **c_dept** et un curseur paramétré **c_emp(p_id)**, avec deux boucles **FOR** :

```
SET sql_mode = ORACLE;

DELIMITER $$
CREATE OR REPLACE PROCEDURE rapport_paie(p_dept NUMBER) AS
  CURSOR c_dept IS
    SELECT dept_id, dept_nom FROM departements
    WHERE p_dept IS NULL OR dept_id = p_dept
    ORDER BY dept_id;
  CURSOR c_emp(p_id NUMBER) IS
    SELECT nom, salaire, prime FROM employes
    WHERE dept_id = p_id
    ORDER BY salaire DESC;
  v_nb          NUMBER;
  v_total_sal   NUMBER;
  v_total_prime NUMBER;
  v_max         NUMBER;
  v_general     NUMBER := 0;
  v_nb_dept     NUMBER := 0;
  e_dept_inconnu EXCEPTION;
BEGIN
  afficher('===== RAPPORT DE PAIE =====');

  FOR d IN c_dept LOOP
    v_nb_dept := v_nb_dept + 1;
    v_nb := 0;
    v_total_sal := 0;
    v_total_prime := 0;
    v_max := 0;
    -- Affichage des données des employés de ce département
  END LOOP;
END;
```

Les primes NULL sont traitées avec **NVL** pour les totaux et un **CASE** pour l’affichage (“aucune”). La section **EXCEPTION** lève notre exception **e_dept_inconnu** (avec **RAISE**) si aucun département ne correspond, et le **WHEN OTHERS** affiche le code et le message de l’erreur (**SQLCODE** / **SQLERRM**) au lieu de ne rien faire :

```

afficher('>> Département ' || d.dept_id || ' : ' || d.dept_nom);

FOR e IN c_emp(d.dept_id) LOOP
    v_nb := v_nb + 1;
    v_total_sal := v_total_sal + e.salaire;
    v_total_prime := v_total_prime + NVL(e.prime, 0);
    IF e.salaire > v_max THEN
        v_max := e.salaire;
    END IF;
    afficher(' ' || e.nom || ' | salaire ' || e.salaire || ' € | prime '
    || ' || CASE WHEN e.prime IS NULL THEN 'aucune' ELSE e.prime || ' €' END);
END LOOP;

IF v_nb = 0 THEN
    afficher(' (département vide, aucun employé)');
ELSE
    afficher(' Total : ' || v_total_sal || ' € de salaires + ' || v_total_prime
    || ' € de primes | Salaire max : ' || v_max || ' €');
END IF;
v_general := v_general + v_total_sal + v_total_prime;
END LOOP;

IF v_nb_dept = 0 THEN
    RAISE e_dept_inconnu;
END IF;

afficher('=====');
afficher('TOTAL GÉNÉRAL (salaires + primes) : ' || v_general || ' €');
EXCEPTION
    WHEN e_dept_inconnu THEN
        afficher('Erreur : le département ' || p_dept || ' n''existe pas');
    WHEN OTHERS THEN
        afficher('Erreur pendant le rapport (' || SQLCODE || ') : ' || SQLERRM);
END;
$$
DELIMITER ;

TRUNCATE TABLE sortie;
CALL rapport_paie(NULL);
SELECT ligne FROM sortie ORDER BY num;

```

Résultat avec tous les départements :

✓ Affichage des lignes 0 - 22 (total de 23, traitement en 0,0002 seconde(s).)

```
SELECT * FROM `sortie`
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

	num	ligne
☐ Éditer ☐ Copier ☐ Supprimer	1	===== RAPPORT DE PAIE =====
☐ Éditer ☐ Copier ☐ Supprimer	2	>> Département 10 : Compta
☐ Éditer ☐ Copier ☐ Supprimer	3	Martin salaire 3200.00 € prime 500.00 €
☐ Éditer ☐ Copier ☐ Supprimer	4	Durand salaire 2800.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	5	Petit salaire 2800.00 € prime 300.00 €
☐ Éditer ☐ Copier ☐ Supprimer	6	Leroy salaire 1900.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	7	Total : 10700 € de salaires + 800 € de primes Salaire max : 3200 €
☐ Éditer ☐ Copier ☐ Supprimer	8	>> Département 20 : RH
☐ Éditer ☐ Copier ☐ Supprimer	9	Moreau salaire 2500.00 € prime 200.00 €
☐ Éditer ☐ Copier ☐ Supprimer	10	Simon salaire 2100.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	11	Laurent salaire 1700.00 € prime 100.00 €
☐ Éditer ☐ Copier ☐ Supprimer	12	Total : 6300 € de salaires + 300 € de primes Salaire max : 2500 €
☐ Éditer ☐ Copier ☐ Supprimer	13	>> Département 30 : IT
☐ Éditer ☐ Copier ☐ Supprimer	14	Garcia salaire 4100.00 € prime 900.00 €
☐ Éditer ☐ Copier ☐ Supprimer	15	Roux salaire 3600.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	16	Fournier salaire 3000.00 € prime 400.00 €
☐ Éditer ☐ Copier ☐ Supprimer	17	Girard salaire 2200.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	18	Bonnet salaire 1800.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	19	Total : 14700 € de salaires + 1300 € de primes Salaire max : 4100 €
☐ Éditer ☐ Copier ☐ Supprimer	20	>> Département 40 : Archives
☐ Éditer ☐ Copier ☐ Supprimer	21	(département vide, aucun employé)
☐ Éditer ☐ Copier ☐ Supprimer	22	=====
☐ Éditer ☐ Copier ☐ Supprimer	23	TOTAL GÉNÉRAL (salaires + primes) : 34100 €

Avec un seul département (CALL rapport_paie(30)) :

✓ Affichage des lignes 0 - 9 (total de 10, traitement en 0,0002 seconde(s).)

```
SELECT * FROM `sortie`
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

	num	ligne
☐ Éditer ☐ Copier ☐ Supprimer	1	===== RAPPORT DE PAIE =====
☐ Éditer ☐ Copier ☐ Supprimer	2	>> Département 30 : IT
☐ Éditer ☐ Copier ☐ Supprimer	3	Garcia salaire 4100.00 € prime 900.00 €
☐ Éditer ☐ Copier ☐ Supprimer	4	Roux salaire 3600.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	5	Fournier salaire 3000.00 € prime 400.00 €
☐ Éditer ☐ Copier ☐ Supprimer	6	Girard salaire 2200.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	7	Bonnet salaire 1800.00 € prime aucune
☐ Éditer ☐ Copier ☐ Supprimer	8	Total : 14700 € de salaires + 1300 € de primes Salaire max : 4100 €
☐ Éditer ☐ Copier ☐ Supprimer	9	=====
☐ Éditer ☐ Copier ☐ Supprimer	10	TOTAL GÉNÉRAL (salaires + primes) : 16000 €

Avec un département qui n'existe pas, l'exception est bien interceptée :

The screenshot shows a SQL query execution interface. At the top, a status bar indicates 'Affichage des lignes 0 - 1 (total de 2, traitement en 0,0002 seconde(s))'. Below this, the SQL query 'SELECT * FROM `sortie`' is displayed. A toolbar contains buttons for 'Profilage', 'Éditer en ligne', 'Éditer', 'Expliquer SQL', 'Créer le code source PHP', and 'Actualiser'. Below the toolbar, there are options for 'Tout afficher', 'Nombre de lignes' (set to 25), 'Filtrer les lignes' (with a search box), and 'Trier par clé'. A section titled 'Options supplémentaires' is visible. The main area displays a table with two columns: 'num' and 'ligne'. The first row (num 1) contains the text '===== RAPPORT DE PAIE ====='. The second row (num 2) contains the error message 'Erreur : le département 99 n'existe pas', which is highlighted with a red border.

num	ligne
1	===== RAPPORT DE PAIE =====
2	Erreur : le département 99 n'existe pas

Compte rendu final

1. Dans quel cas utiliseriez-vous un curseur explicite plutôt qu'un SELECT INTO ?

Quand la requête peut renvoyer plusieurs lignes (ou aucune) et qu'on veut les traiter une par une, par exemple pour faire un rapport comme dans l'exercice 7 ou modifier chaque ligne avec FOR UPDATE. Le SELECT INTO c'est seulement quand on est sûr d'avoir une seule ligne : recherche par clé primaire, COUNT(*), SUM..., comme dans les fonctions de la partie 1.

2. Quelle erreur sur les curseurs vous a le plus surpris, et pourquoi ?

Le **#1326 Cursor is not open** quand on lit %ROWCOUNT après le CLOSE ou après une boucle FOR. On pensait que le curseur gardait le nombre de lignes lues, en fait une fois fermé on ne peut plus rien lui demander à part %ISOPEN. Ça nous a obligé à utiliser un compteur dans l'exercice 5.

3. Une chose qui n'a pas marché du premier coup, et comment vous l'avez résolue.

Le ROLLBACK de l'exercice 1 : on pensait annuler l'UPDATE mais phpMyAdmin est en autocommit, donc les salaires du département 20 ont vraiment augmenté de 1 €. On l'a vu en refaisant un SELECT, on a remis les salaires avec un UPDATE ... salaire - 1 et on a ajouté **SET autocommit = 0** au début des scripts qui font des ROLLBACK ou des verrous. Il y a eu aussi DBMS_OUTPUT qui n'existe pas, réglé avec la table sortie et la procédure afficher().

Conclusion

Ce TP nous a permis de comprendre comment fonctionnent les **curseurs** : le curseur implicite SQL et les limites du SELECT INTO, le cycle d'un curseur explicite (OPEN, FETCH, CLOSE) et ses attributs, les trois façons de boucler, les curseurs paramétrés et le verrouillage avec **FOR UPDATE**.

Comme on utilise phpMyAdmin et pas Oracle, il a fallu adapter pas mal de choses (mode Oracle de MariaDB, table sortie à la place de DBMS_OUTPUT, autocommit, pas de WHERE CURRENT OF ni de NOWAIT dans les blocs), mais ça nous a aussi montré les différences entre les deux SGBD.